

Emotion Detection with CNN / ViT: What Model Presents Better Efficiency for Classification

HYUNWOO OH, 17243198 and OMAR ATTAALLA, 79825434, Project Group 5

This report presents the application of Convolutional Neural Networks (CNNs) and Vision Transformer (ViT) models to the emotion detection task. The CNN model was implemented using the TensorFlow library with Keras, while the ViT models, including DeiT variants, were trained using PyTorch. The dataset was preprocessed to align with the input requirements of both architectures, with images resized to appropriate dimensions for each model.

The results suggest that CNN significantly outperforms ViT for this task, achieving an accuracy of 98%, compared to the maximum accuracy of 53.97% attained by the ViT models. This performance gap highlights the reliability of CNNs for traditional image classification tasks like emotion detection. However, the study also reveals limitations in ViT's ability to generalize on smaller datasets and suggests potential overfitting issues.

Future work will focus on optimizing ViT models, exploring domain-specific pretraining, and leveraging multi-modal approaches to improve their applicability in emotion detection tasks.

1 Introduction

Computer vision using machine learning models is rapidly advancing and being adopted in a wide range of applications. Among the prominent architectures, Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs) are widely used for tackling image-based tasks. While CNNs have long been the dominant models for image classification tasks, ViTs have emerged as strong competitors, leveraging unique mechanisms such as tokenization of image patches and self-attention.

These distinctive attributes of ViTs open new possibilities for developing embedding-based multi-modal AI architectures. Models like ImageBind [3] and OpenCLIP [2] exemplify this by integrating text, images, and audio into a unified embedding space, enabling seamless cross-modal interactions.

Despite the promise of ViTs, CNNs remain a proven and reliable architecture for image classification tasks. Over the years, many CNN-based models have achieved superhuman accuracy in various benchmarks, cementing their place as the go-to choice for image-related applications. Given this context, selecting the appropriate architecture depends heavily on application-specific constraints such as accuracy, computational throughput, and memory efficiency.

In this report, we investigate and compare the performance of CNNs and ViTs in the context of emotion detection, a challenging image classification task. By evaluating various CNN and ViT models, we aim to provide insights into their relative strengths and weaknesses, ultimately guiding model selection for future applications.

2 Convolutional Neural Network

Our first approach to constructing a model that can predict emotion from an image is using convolutional neural networks. Convolutional neural networks have grown in popularity when solving image based tasks. It's ability to extract features from images has made it an extremely popular setup for image classification, detection, and segmentation. This section will describe our approach to building our own convolutional neural network that can solve the emotion detection task.

2.1 Setup

Because of Python's dominance of the machine learning space, we chose to use it as our primary language. To construct our model, we chose to use the popular library Tensorflow. In particular, we chose to use Keras to construct and test our convolutional neural network. We also chose to use Pandas for constructing our datasets

from the .csv file given to us and Matplotlib for plotting the training process.

In order to train our convolutional neural network, we needed some sort of compute. This is one issue we faced since we are compute constrained by the compute that we have available. Using our local machine, we accelerated the training process slightly by using Nvidia Cuda's CUDNN combined with an Nvidia RTX 3070. This accelerated the training process drastically, compared to CPU-only training, but it still was a limitation in our system and prohibited us from doing further testing and making changes to our model.

2.2 Pre-processing

Before building our model, we needed to extract training and validation datasets, preprocess each image, and extract the labels for each image. This process was made easy by using Pandas. The dataset we used (muxspace/-facial_expressions) included approximately 13,000 350x350 images and their labels. As such, we used a random process to assign 20% of the dataset to the validation set and the remaining 80% to the training set. Since every image in the dataset is already in the desired size, we did not need to resize each image in the pre-processing step. This step differed in each iteration of the CNN. In the final CNN, we included data augmentation in our pre-processing step that was not otherwise included in previous iterations. This data augmentation included rotation, zoom, and rescaling.

2.3 First iteration CNN

Our first iteration CNN involved a simple model with an input layer with the desired image size (350,350), a Conv2D layer with 32 filters, a 3x3 convolutional window and using the ReLU activation function. Then, we have a flatten layer which outputs a one-dimensional output for the convolutional layer output. Finally, our output layer is a dense layer that uses softmax to compute the classification probabilities based on our label map. The keras equivalent code for constructing this model is shown below:

```
model = models.Sequential([
    layers.Input(shape=(350, 350, 3)),
    layers.Conv2D(32, kernel_size=(3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(len(label_map), activation='softmax'),
])
```

This model is simple since it only involves one convolutional layer. In a later section, we will describe the training and testing performance of this model and if it is enough for our application (emotion detection).

2.4 Final iteration CNN

After many iterations and attempts at many different layers combinations, the model that performed best involved multiple convolutional layers and the introduction of max pooling layers. The max pooling layer is particularly useful since it is a dimensionality reduction method that allows us to focus on areas of the image that matter the most [1]. This dimensionality reduction reduced the overall computation load required by our model, which is useful in our case because we are compute-constrained. The code for the final iteration CNN is shown below:

```
model = models.Sequential([
    layers.Input(shape=(350, 350, 3)),
    layers.Conv2D(32, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D(pool_size=(2, 2)),
```

```
layers.Conv2D(64, kernel_size=(3, 3), activation='relu'),  
layers.MaxPooling2D(pool_size=(2, 2)),  
layers.Conv2D(128, kernel_size=(3, 3), activation='relu'),  
layers.MaxPooling2D(pool_size=(2, 2)),  
layers.Flatten(),  
layers.Dense(len(label_map), activation='softmax'),  
)
```

This model is much more complex than the previous iteration CNN and involves 3 CNN layers with 32, 64, and 128 filters respectively. After each Conv2D, we introduce a max pooling layer with a downscale factor of (2,2). As previously explained, this reduces the compute required to train our model.

2.5 Training Results

To test the performance of this model, we used Kera's training framework that allowed us to keep track of the loss, validation loss, accuracy, and validation accuracy. We also introduced early callbacks to ensure that we keep track of the best model. In particular, we used early stopping, and reducing the learning rate on plateau. The code for these callbacks are shown below:

```
callbacks = [  
    EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=True),  
    ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=2, min_lr=1e-5)  
]
```

Training was based on the Adam optimizer which is a stochastic gradient descent method that uses adaptive estimation to alter the learning rate dynamically. The computed loss used the sparse categorical cross entropy function since we have multiple classes (emotions). The code for training is shown below:

```
model.compile(  
    optimizer=keras.optimizers.Adam(0.001),  
    loss=keras.losses.SparseCategoricalCrossentropy(),  
    metrics=[keras.metrics.SparseCategoricalAccuracy(name="acc")],  
)  
  
history = model.fit(train_dataset, epochs=10,  
                    validation_data=val_dataset,  
                    callbacks=callbacks)
```

2.5.1 First iteration CNN Results. Our first iteration CNN was previously shown to be simple as it only used on convolutional layer. To plot the results of training this model, we used matplotlib. The results of training this model is shown below in Figure 1: Given how simple the model was, it failed to capture the features from the images to make good predictions. Overall, we only saw a maximum training accuracy of approximately 59%. However, the validation accuracy can be seen jumping around with no clear trend. From this, we can conclude that the first iteration model is too simple and fails to extract the features from the images. As such, it is underfitting and in the next iterations, we chose to expand the complexity of the model. The average time spent on an epoch in this case (using our setup) was only 22s.

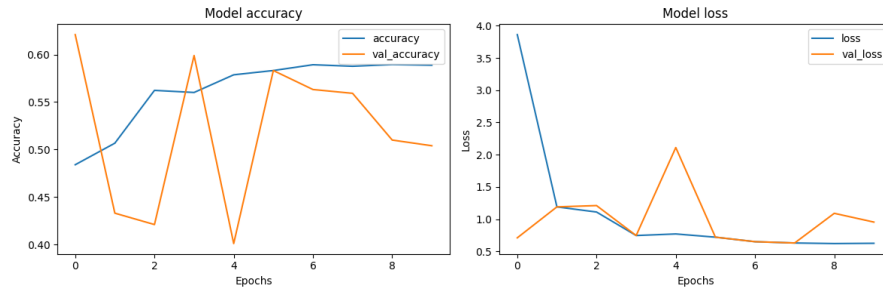


Fig. 1. First Iteration CNN Loss & Accuracy

2.5.2 Final iteration CNN Results. In the final iteration CNN, we introduced new layers and we introduced data augmentation. As such, the model complexity increased and the overall average time for an epoch increased to 2 minutes and 36 seconds. However, we chose to increase the number of epochs to 20 to capture the overall trend and to extract the best possible model. The results of training this model are shown below in Figure 2:

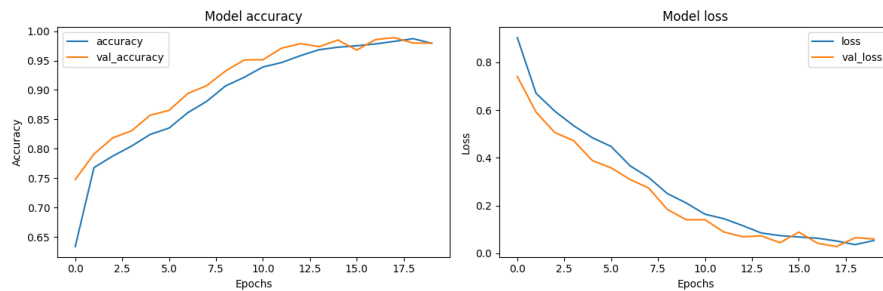


Fig. 2. Final Iteration CNN Loss & Accuracy

We can see a clear trend of both the validation and training accuracy. Both the training and validation accuracies plateaued at approximately 98% which is where we want the model to be. However, we did not expect the validation accuracy to exceed the training accuracy. One possible reason for this could be that since we use data augmentation in the training process, but not the validation process. Therefore, the training data set is more difficult to predict than the validation data set.

3 Vision Transformer

Our second approach is to apply ViT for emotion detection task from the given image. ViT is an emerging model which is actively researched nowadays. Unlike the original transformer architecture for natural language processing (NLP), ViT divides the images into multiple patches with grids and considers the image patches as token and convert to embedding for each patch with adding positional arguments into it. This section describes how we applied ViT to emotional detection task.

3.1 Setup

Similar to the CNN, we chose Python language, but the pyTorch framework is selected as most of the ViT models are dominantly implemented with pyTorch. We choose one of the ViT-variant transformer model, data-efficient

transformer (DeiT), to adopt ViT to the emotion detection task. We executed the training process using Nvidia RTX 4080.

3.2 Pre-processing

The training and validation sets are split using the `train_test_split` function in scikit-learn library. We set the train/validation ration as 0.9 : 0.1. Every images have been resized to 224×224. This ensures the images to match the input image size on the popular ViT models, which divides 224×224 to 14×14 number of 16×16 patches.

3.3 DeiT

The DeiT model configurations we have chose to apply the emotion detection task are shown in table 1. We developed the DeiT-atom model for emotion detection. DeiT-tiny and DeiT-small is a pre-defined model from the authors of DeiT paper. This is two smallest model from the original DeiT implementation. However, we analyzed that it is still large model for the given dataset, which contains 13936 images, divided into 12539 training and 1397 validation sets. Therefore, we created the DeiT-atom model, which is even smaller than the DeiT-tiny model.

Table 1. DeiT model configurations

	Patch Size	Embedding Dimension	Head Count	MLP Ratio	Parameter Count
DeiT-atom	16×16	96	3	4.0	1.44 M
DeiT-tiny	16×16	192	3	4.0	5.53 M
DeiT-small	16×16	384	6	4.0	21.67M

The training parameters are shown in table 2. We applied adam optimizer with learning rate of 0.0005 (5e-4). The epoch count is set to 30. We changed the batch size configurations for each attempts to analyze the impact of the stochastic operation of weight update.

Table 2. Training parameter configurations

Batch Size	Learning Rate	Optimizer	Epochs
4, 16, 64, 256	0.0005	AdamOptimizer	30

3.4 Results

The results of the ViT-based emotion detection task are summarized in Table 3. We trained the DeiT-atom, DeiT-tiny, and DeiT-small models using the specified configurations to analyze their performance on the validation set. Each configuration represents different batch sizes: 4, 16, 64, and 256, allowing us to observe how batch size influences the model's performance.

As shown in Table 3, the accuracy values are reported for each batch size across the three models. The DeiT-atom model demonstrated the lowest accuracy but offered a significant reduction in parameter count, making it suitable for resource-constrained environments. Conversely, the DeiT-small model achieved the highest accuracy, albeit with a considerably larger parameter count.

Table 3. Accuracy breakdowns of the DeiT

Configuration	DeiT-atom	DeiT-tiny	DeiT-small
Accuracy (Val.)	51.11/51.40/51.61/51.97	51.18/51.18/51.18/51.18	53.97/52.76/52.11/51.32

3.5 Discussion

From the results, it is evident that the batch size does not significantly impact the accuracy of the DeiT models for the given dataset. All models achieved relatively stable performance across different batch sizes, suggesting that the optimization process is not heavily influenced by stochastic variations in gradient updates.

The DeiT-atom model, with its minimal parameter count, is particularly advantageous for scenarios where computational resources or inference time is a constraint. However, its lower accuracy might limit its applicability in scenarios demanding high precision. On the other hand, the DeiT-small model achieves better accuracy, making it suitable for environments where computational resources are not as restricted.

Despite these observations, the overall accuracy across all models remains relatively low, indicating challenges in applying ViT-based approaches to the emotion detection task with the current dataset. Potential reasons include:

- Limited dataset size (13,936 images), which might not be sufficient for training transformer-based models effectively.
- Complexity of emotion detection, which often requires understanding subtle visual cues.
- Lack of pretraining specific to emotion detection, as the ViT models used are pre-trained on generic datasets.

Future work could focus on augmenting the dataset size, employing domain-specific pretraining, or combining ViT with other modalities (e.g., text or audio) to enhance performance in emotion detection tasks.

4 Conclusion

In this report, we applied and compared Convolutional Neural Networks (CNNs) and Vision Transformer (ViT) models for the task of emotion detection. For the CNN approach, we designed a custom model that accepts 350×350 images for classification. For the ViT approach, we experimented with DeiT models, utilizing both pre-defined and customized configurations to evaluate their effectiveness.

The results demonstrate that the CNN model significantly outperforms the ViT models, achieving an impressive accuracy score of 98%. In contrast, the ViT models, even with varied configurations, achieved a maximum accuracy of only 53.97%. This suggests that for traditional image classification tasks like emotion detection, CNNs currently serve as a more suitable choice compared to ViT-based models.

However, the performance gap raises important questions about the potential overfitting tendencies in ViT. Interestingly, even when the parameter count of the ViT models was increased, the accuracy did not improve. This indicates the need for further exploration to understand the behavior of ViT on smaller datasets and its limitations in emotion detection.

In the future, we plan to:

- Experiment with smaller and more optimized ViT models to mitigate overfitting.
- Conduct a fairer comparison by aligning the model sizes and complexities of CNNs and ViTs.
- Explore domain-specific pretraining and data augmentation techniques to enhance the performance of ViT.
- Investigate multimodal approaches that integrate ViT with other data modalities, such as text or audio, to complement its capabilities.

References

- [1] S.; Bai Q.; Yang J.; Jiang S.; Miao Y Chen, L.; Li. 2021. Review of Image Classification Algorithms Based on Convolutional Neural Networks. 6.
- [2] Mehdi Cherti, Romain Beaumont, Ross Wightman, Mitchell Wortsman, Gabriel Ilharco, Cade Gordon, Christoph Schuhmann, Ludwig Schmidt, and Jenia Jitsev. 2023. Reproducible scaling laws for contrastive language-image learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2818–2829.
- [3] Rohit Girdhar, Alaaeldin El-Nouby, Zhuang Liu, Mannat Singh, Kalyan Vasudev Alwala, Armand Joulin, and Ishan Misra. 2023. ImageBind: One Embedding Space To Bind Them All. In *CVPR*.